Password Generator & Strength Checker

Submitted By

Ardra Rajesh

Internship

CODEVEDX Python Programming Internship

Department

B.Tech Computer Science & Engineering

Date

27 June 2026

Table of Contents

1. Introduction
2. Objectives
3. Features
4. Technologies Used
5. Python Modules Used
6. Project Workflow
7. Project Structure
8. Screenshots
9. Sample Output
10. Error Handling
11. Future Enhancements
12. Conclusion

Introduction

Passwords play a vital role in protecting personal and organizational data from unauthorized access. Weak passwords are one of the leading causes of security breaches, making it essential to generate strong and unpredictable passwords while also evaluating their strength.

The **Password Generator & Strength Checker** is a Python-based console application designed to generate secure passwords using Python's **secrets** module, which provides cryptographically secure random values. The application also evaluates the strength of user-provided passwords based on multiple security criteria such as length, character diversity, common weak patterns, repeated characters, and sequential numbers.

The system provides a user-friendly menu-driven interface for generating single or multiple passwords, checking password strength, and maintaining a secure history of password generation activities. To ensure privacy, the application stores only password metadata such as generation time, password length, score, and strength rating, while never saving the actual passwords.

This project helped me understand secure programming practices, modular programming, file handling, regular expressions, input validation, exception handling, and cybersecurity concepts using Python.

Objectives

The main objectives of this project are:

- To develop a secure password generation system using Python.
- To generate cryptographically secure passwords.
- To evaluate password strength based on multiple security factors.
- To provide useful feedback for improving weak passwords.
- To generate multiple unique passwords simultaneously.
- To maintain password history securely without storing actual passwords.
- To improve programming skills using functions, modules, JSON file handling, and exception handling.
- To understand secure software development practices using Python.

Features

Secure Password Generation

- Generate cryptographically secure passwords.
- Use Python's **secrets** module for randomness.
- Guarantee strong password generation.

Customizable Password Options

- Select password length.
- Include uppercase letters.
- Include lowercase letters.
- Include numbers.
- Include special characters.
- Ensure every selected category appears at least once.

Multiple Password Generation

- Generate up to 20 unique passwords at once.
- Prevent duplicate passwords.

Password Strength Analysis

- Analyze password length.
- Detect uppercase, lowercase, numbers, and symbols.
- Identify common weak passwords.
- Detect repeated characters.
- Detect sequential number patterns.
- Display password score.
- Display strength rating.
- Provide improvement suggestions.
- Show a visual progress bar.

Password History

- Save generation history locally.
- Store timestamp.
- Store password length.
- Store strength rating.
- Store security score.
- Never store actual passwords.

Additional Features

- User-friendly menu-driven interface.
- Input validation.
- Exception handling.
- Automatic JSON file creation.
- Secure metadata storage.

Technologies Used

Technology	Purpose
Python 3	Programming Language
JSON	Local Data Storage
VS Code	Code Editor
Git	Version Control
GitHub	Project Repository

Python Modules Used

secrets

Used to generate cryptographically secure random passwords.

string

Provides uppercase letters, lowercase letters, digits, and special characters.

re

Used to analyze password strength using regular expressions.

json

Stores password history in JSON format.

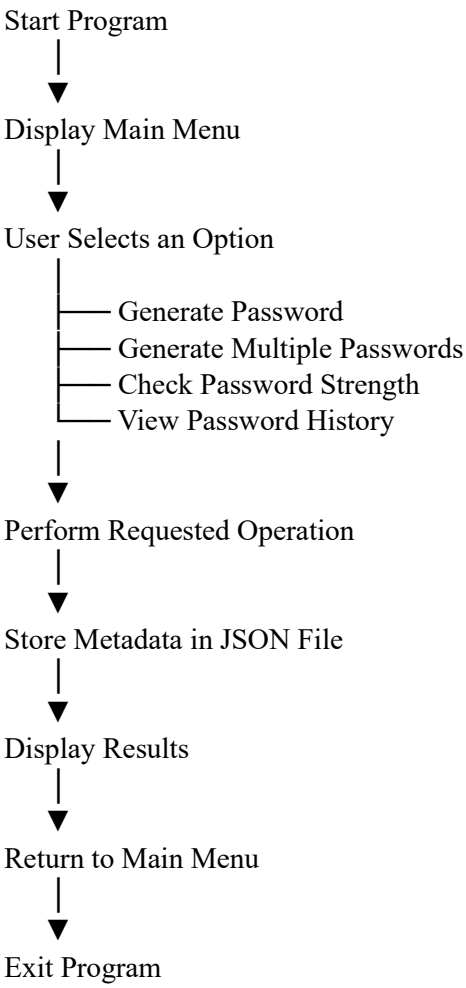
os

Checks file existence and manages history files.

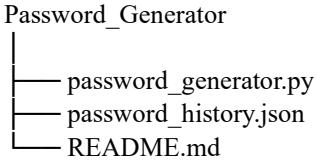
datetime

Records timestamps for password generation history.

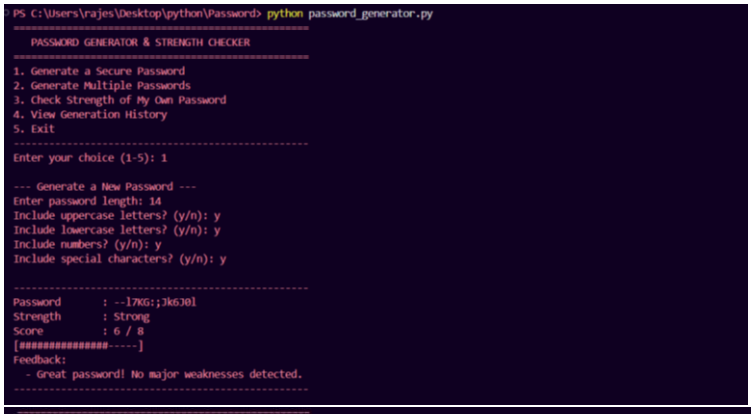
Project Workflow

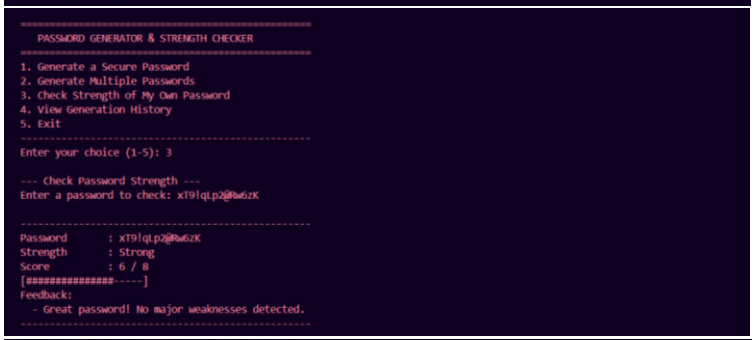


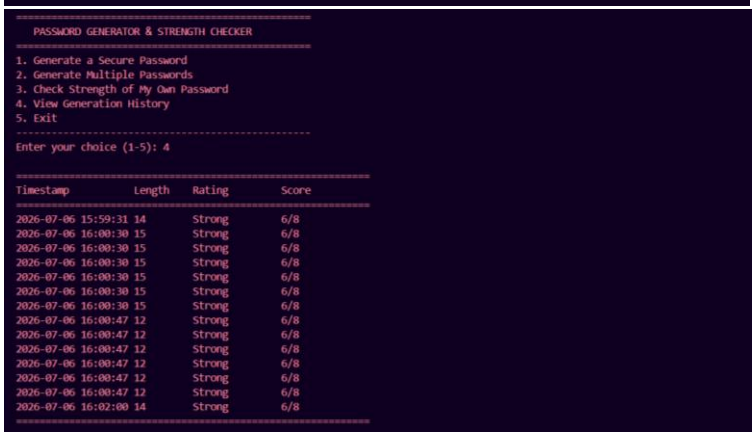
Project Structure



Screenshots







Sample Output

========== PASSWORD GENERATOR & STRENGTH CHECKER ==========

1. Generate a Secure Password
2. Generate Multiple Passwords
3. Check Strength of My Own Password
4. View Generation History
5. Exit

Enter your choice: 3

Password : xT9!qLp2@Rw6zK

Strength : Very Strong

Score : 8 / 8

Feedback

• Great password! No major weaknesses detected.

Error Handling

To make the application secure and reliable, several validation and exception handling techniques have been implemented.

The system checks for:

- Empty password input
- Invalid menu selections
- Invalid password length
- No character type selected
- Password length smaller than selected character categories
- Invalid numeric input
- Missing history JSON file (created automatically)
- Corrupted JSON files
- Duplicate passwords during batch generation

These validations improve the security, stability, and user experience of the application.

Future Enhancements

The project can be further improved by adding:

- Graphical User Interface (GUI) using Tkinter
- Password export in encrypted format
- Password copy-to-clipboard feature
- Passphrase generation
- Exclude similar-looking characters
- Password expiration reminders
- Cloud backup for password history
- Machine learning–based password risk prediction
- Multi-language support

Conclusion

The **Password Generator & Strength Checker** was developed to gain practical experience in secure Python programming while addressing the real-world need for strong password management. Through this project, I learned how to generate cryptographically secure passwords using Python's secrets module, evaluate password strength with regular expressions, manage data using JSON files, validate user input, and implement exception handling.

Working on this project also enhanced my understanding of modular programming, file handling, cybersecurity principles, and secure application development. Features such as customizable password generation, password strength analysis, secure history management, and multiple password generation made the application practical and improved my problem-solving skills.

Overall, this project strengthened my confidence in building secure Python applications and provided valuable hands-on experience that will be useful in future software development and cybersecurity-related projects.